ALL: A Programming Model for Composable Accelerator Design

Niansong Zhang, Hongzheng Chen, Zhiru Zhang

Cornell University

RAIC Tutorial @ ASPLOS'26

Cornell University

ZHANG

Complexity of Hardware Accelerator Design

```
void systolic_tile(int8_t A_tile[2][768],
 int8_t B_tile[768][2],
 int8_t C_tile[2][2]) {
 #pragma dataflow
 hls::stream<int8_t> A_fifo[2][3], B_fifo[2][3];
 #pragma stream variable=A/B_fifo depth=3
 #pragma partition variable=A/B/C_tile complete dim=1
 for (int k4 = 0; k4 < 768; k4++) {
  for (int m = 0; m < 2; m++) {
   int8_t v105 = A_tile[m][k4];
   A_fifo[m][0].write(v105);}
   // ... write B_fifo
  }}
 for (int Ti = 0; Ti < 2; ++Ti) {
 #pragma HLS unroll
  for (int Tj = 0; Tj < 2; ++Tj) {
  #pragma HLS unroll
   // ... load A/B_fifo
   PE_kernel(A_in, A_out, B_in, B_out, C, Ti, Tj);
}}}

void matmul(int8_t A[512][768], int8_t B[768][768],
 int8_t C[512][768]
) {
 int8_t local_A[2][768], local_B[768][2], local_C[2][2];
 for (int mi = 0; mi < 256; mi++) {
  for (int ni = 0; ni < 384; ni++) {
   // ... load A, B
   systolic_tile(local_A, local_B, local_C);
}}}
```

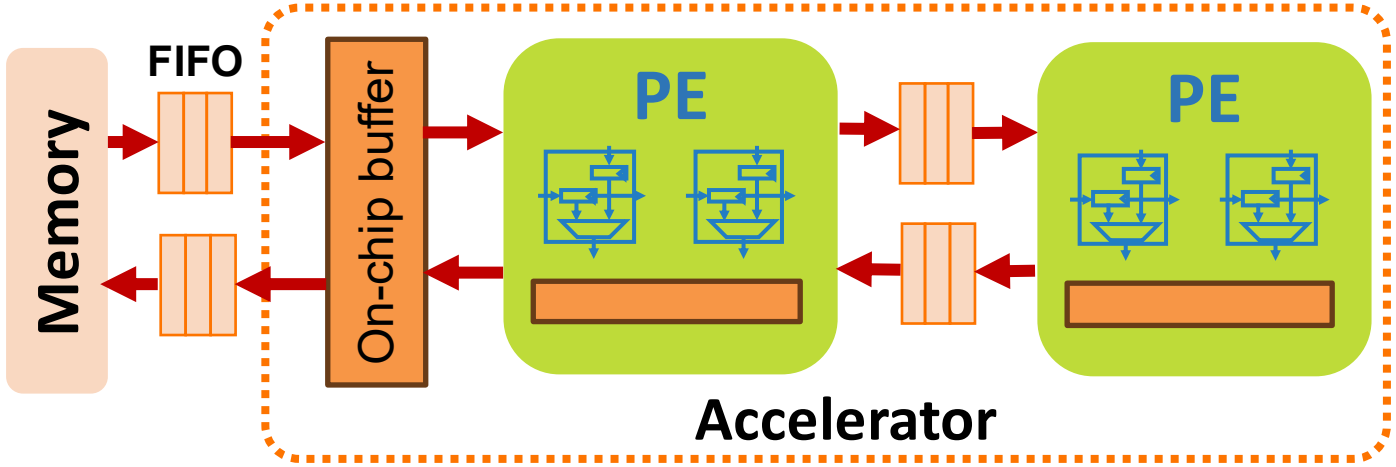
Communication Customization

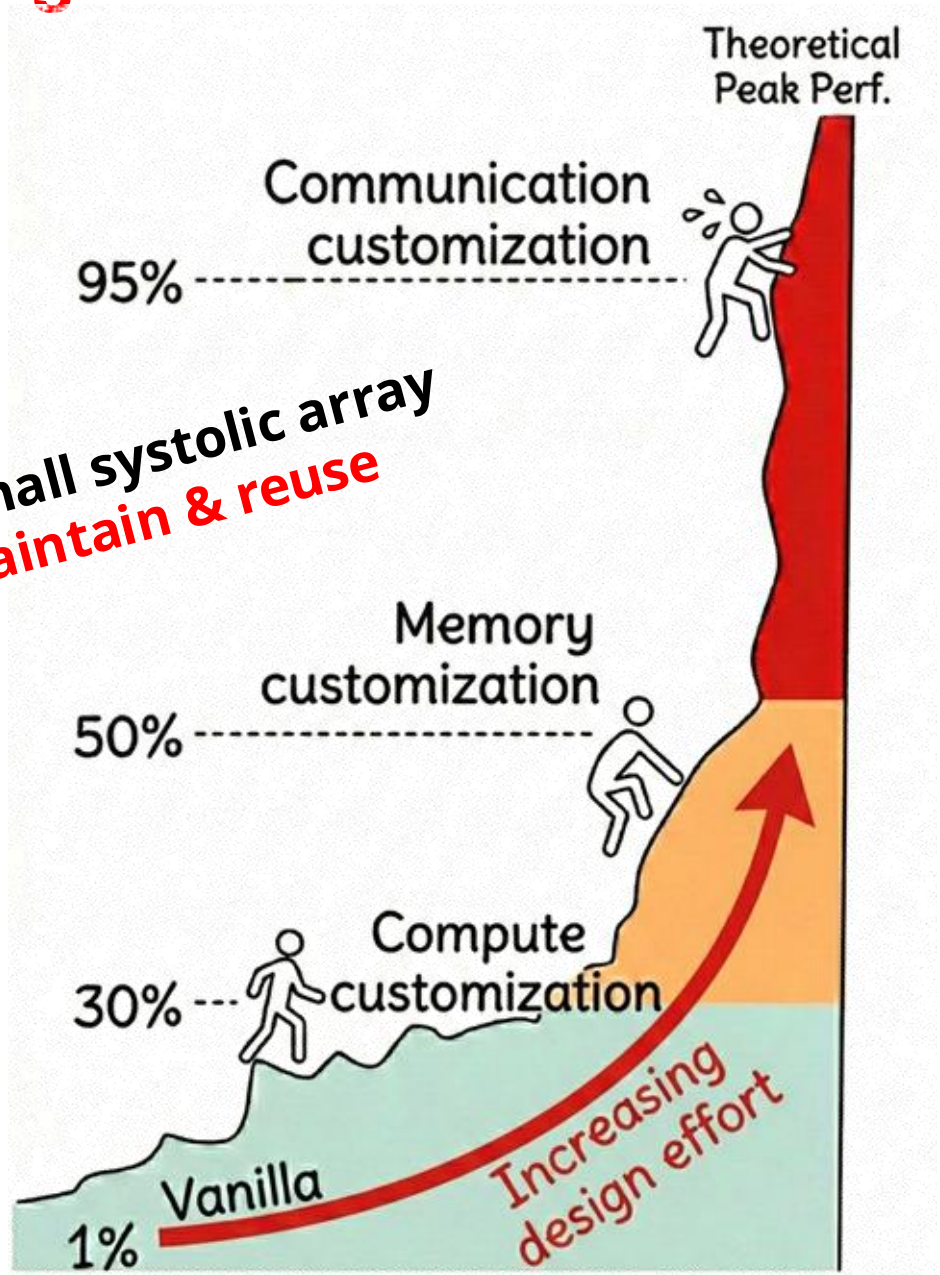
Memory Customization

Compute Customization

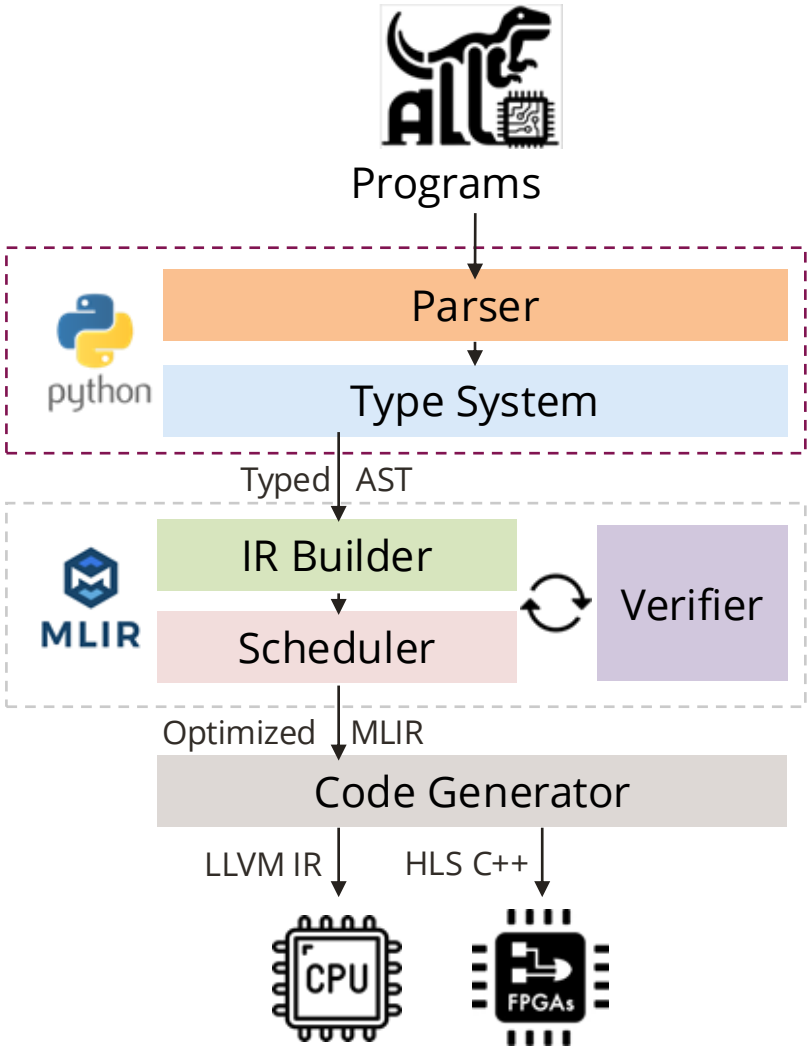
~500 lines of HLS code for a small systolic array

vendor-specific, hard to maintain & reuse

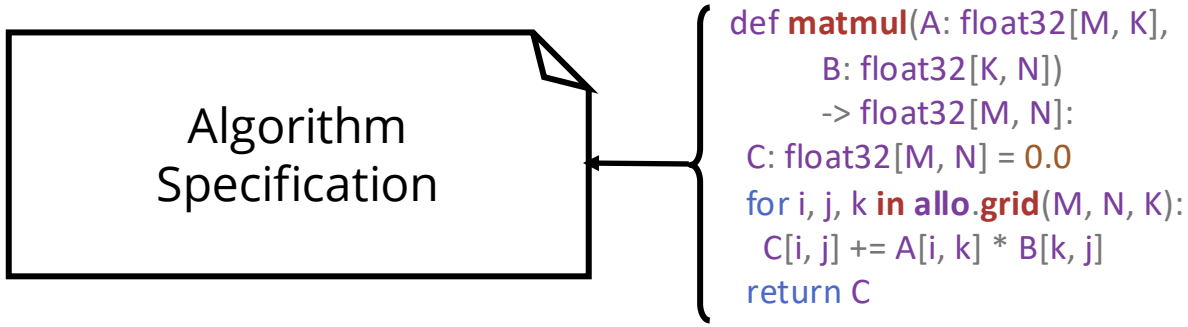




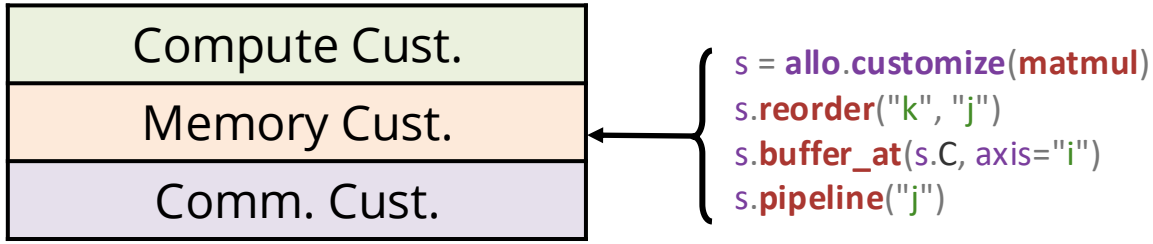
Allo Accelerator Design Language (ADL) and Compiler



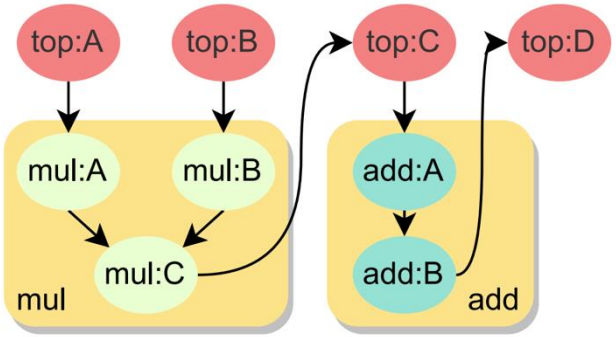
Pythonic: No need to learn a new DSL!



Verifiable decoupled customizations



Type-safe composition



Transforming GEMM to Systolic Array

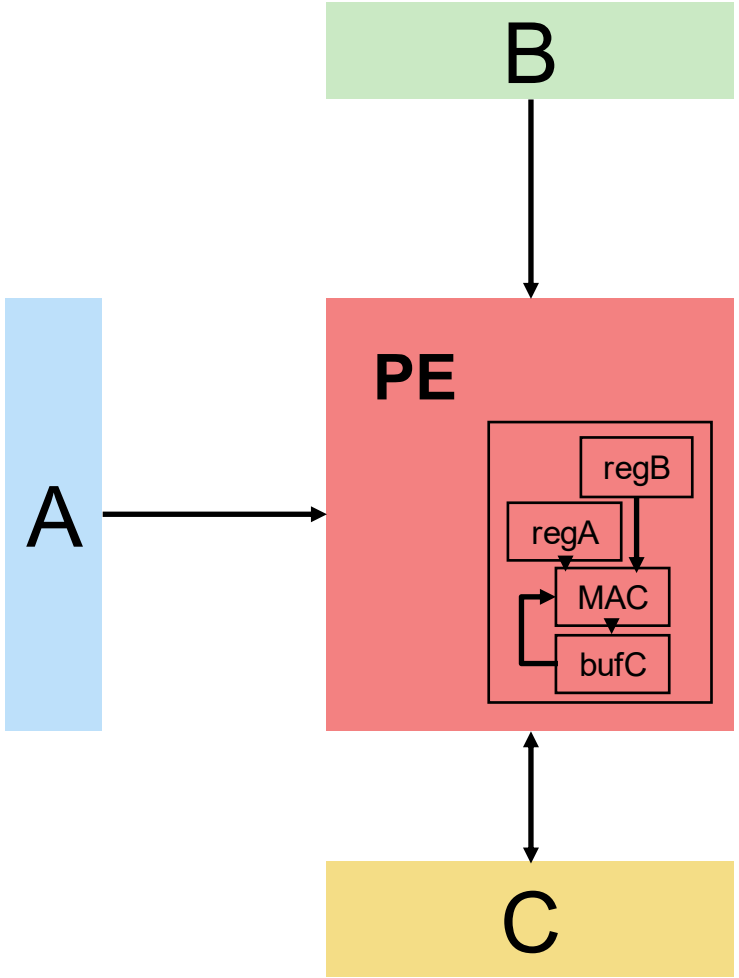
➔ Algorithm specification

```
def matmul(A: int8[M, K],
           B: int8[K, N],
           C: int16[M, N]):
  for i, j in allo.grid(M, N, "PE"):
    for k in range(K):
      C[i, j] += A[i, k] * B[k, j]
```

➔ Schedule construction

```
s = allo.customize(matmul)
```

➔ Architectural Diagram



* Schedule: A sequence of customization primitives

Transforming GEMM to Systolic Array

➔ Algorithm specification

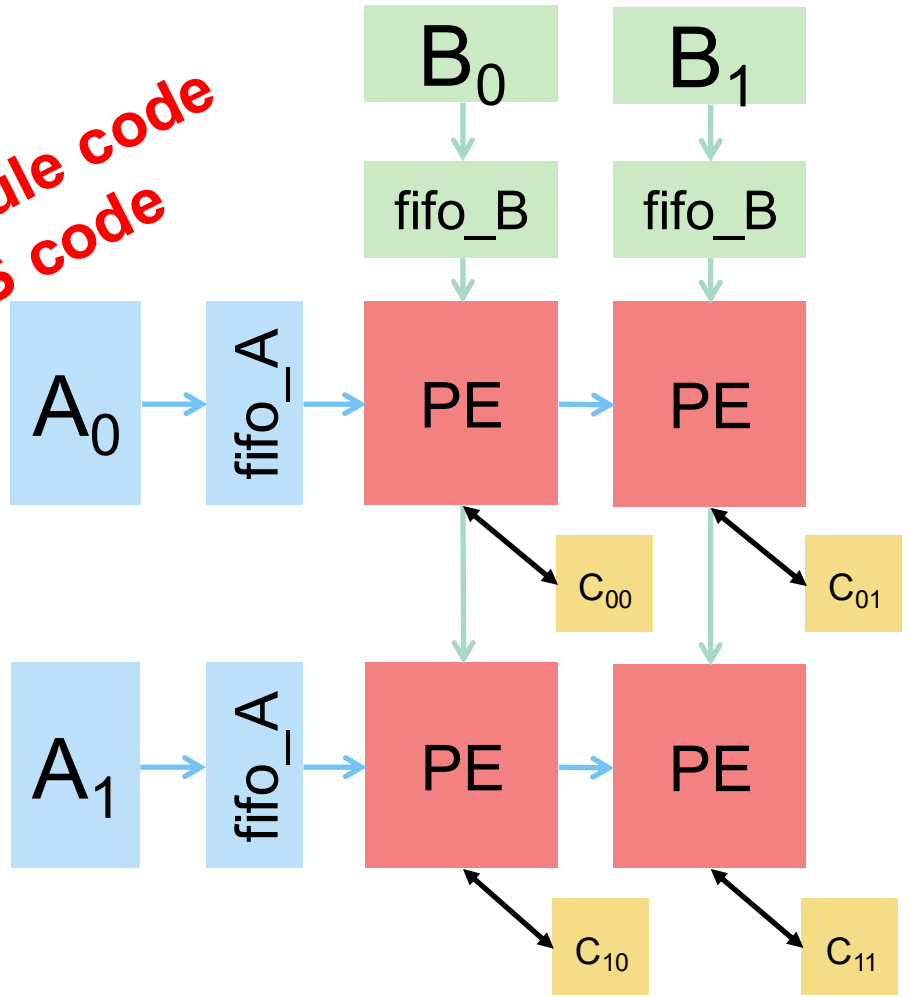
```
def matmul(A: int8[M, K],
           B: int8[K, N],
           C: int16[M, N]):
  for i, j in allo.grid(M, N, "PE"):
    for k in range(K):
      C[i, j] += A[i, k] * B[k, j]
```

➔ Schedule construction

```
s = allo.customize(matmul)
buf_A = s.buffer_at(s.A, "j")
buf_B = s.buffer_at(s.B, "j")
pe = s.unfold("PE", axis=[0, 1],
              factor=[M, N])
s.partition(s.A, dim=0)
s.partition(s.B, dim=1)
s.partition(s.C, dim=[0, 1])
s.relay(buf_A, pe, axis=0, depth=M + 1)
s.relay(buf_B, pe, axis=1, depth=N + 1)
```

Only 8 lines of schedule code vs ~500 lines HLS code

➔ Architectural Diagram



* Schedule: A sequence of customization primitives

Transforming GEMM to Systolic Array

➔ Algorithm specification

```
def matmul(A: int8[M, K],
           B: int8[K, N],
           C: int16[M, N]):
  for i, j in allo.grid(M, N, "PE"):
    for k in range(K):
      C[i, j] += A[i, k] * B[k, j]
```

➔ Schedule construction

```
s = allo.customize(matmul)
buf_A = s.buffer_at(s.A, "j")
buf_B = s.buffer_at(s.B, "j")
pe = s.unfold("PE", axis=[0, 1],
              factor=[M, N])
s.partition(s.A, dim=0)
s.partition(s.B, dim=1)
s.partition(s.C, dim=[0, 1])
s.relay(buf_A, pe, axis=0, depth=M + 1)
s.relay(buf_B, pe, axis=1, depth=N + 1)
```

➔ CPU Simulation

```
mod = s.build(target="llvm")

A = np.random.randint(-8, 8, size=(M, K)) \
    .astype(np.int8)
B = np.random.randint(-8, 8, size=(K, N)) \
    .astype(np.int8)
C = np.zeros((M, N), dtype=np.int16)
mod(A, B, C)
np.testing.assert_allclose(C, A @ B, atol=1e-3)
```

Transforming GEMM to Systolic Array

➔ Algorithm specification

python

```
def matmul(A: int8[M, K],
           B: int8[K, N],
           C: int16[M, N]):
  for i, j in allo.grid(M, N, "PE"):
    for k in range(K):
      C[i, j] += A[i, k] * B[k, j]
```

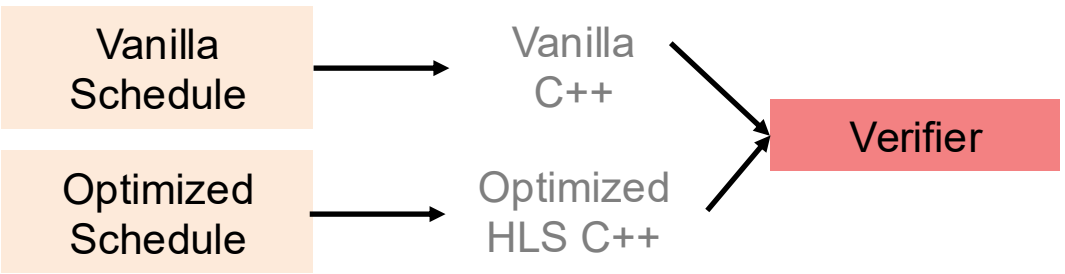
➔ Schedule construction

```
s = allo.customize(matmul)
buf_A = s.buffer_at(s.A, "j")
buf_B = s.buffer_at(s.B, "j")
pe = s.unfold("PE", axis=[0, 1],
              factor=[M, N])
s.partition(s.A, dim=0)
s.partition(s.B, dim=1)
s.partition(s.C, dim=[0, 1])
s.relay(buf_A, pe, axis=0, depth=M + 1)
s.relay(buf_B, pe, axis=1, depth=N + 1)
```

➔ Formal Verification [FPGA'24 Best Paper*]

CPU

```
mod = s.verify()
```



- A **formal equivalence checker** of source-to-source HLS transformations via symbolic execution
  - Support statically interpretable control-flow (SICF)
- Verification in time/space linear w.r.t. OPs executed
  - ~500K Ops/sec in verification throughput
  - A complex **64x64 systolic array verified in 16 minutes**

* L.-N. Pouchet et al., "Formal Verification of Source-to-Source Transformation for HLS", FPGA 2024 (Best Paper Award)

Transforming GEMM to Systolic Array

➔ Algorithm specification

```
def matmul(A: int8[M, K],
           B: int8[K, N],
           C: int16[M, N]):
  for i, j in allo.grid(M, N, "PE"):
    for k in range(K):
      C[i, j] += A[i, k] * B[k, j]
```

➔ Bitstream generation

```
mod = s.build(target="vitis_hls",
              mode="hw",
              project="systolic.prj")

# Automatically invoke the HLS toolchain
mod(A, B, C)
```

First FFN layer in BERT-base (512, 768)x(768, 3072) w/ 16x16 SA

	Latency (ms)	BRAM	DSP	FF	LUT
Allo	**15.73**	0 (0%)	**128 (1%)**	79969 (3%)	244439 (18%)
AutoSA	15.71	514 (12%)	256 (2%)	100138 (3%)	244032 (18%)

Same level of performance but much lower resource usage

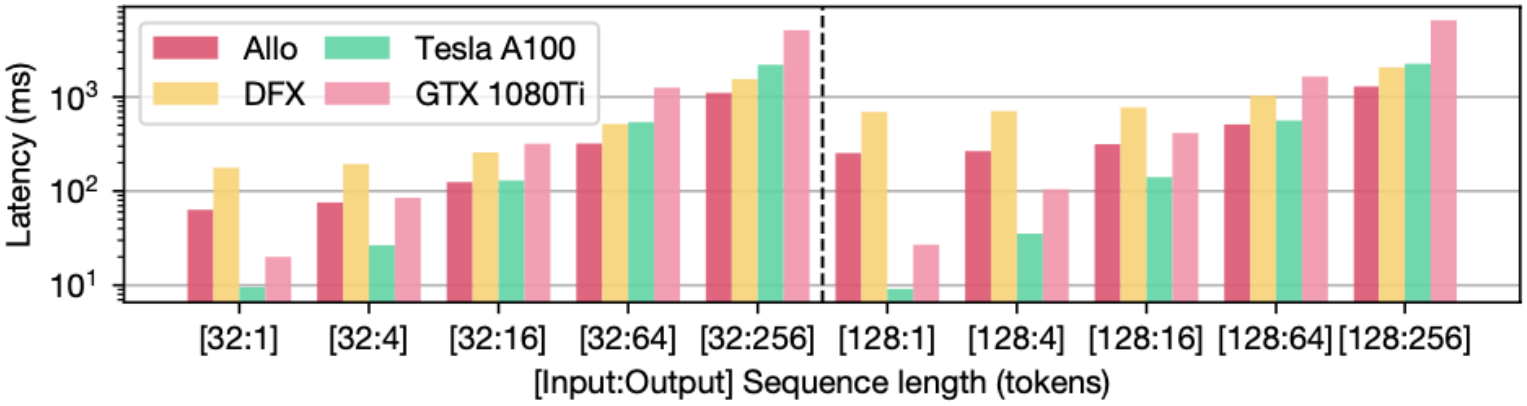
* J. Wang et al., “AutoSA: A Polyhedral Compiler for High-Performance Systolic Arrays on FPGA”, FPGA, 2021.

LLM Accelerator Evaluation

- GPT2: single-batch, low-latency, generative inference settings
  - Full design running on an AMD Alveo U280 FPGA (16nm) at 250MHz
  - 2.2x speedup in prefill stage compared to DFX [MICRO'22] (an FPGA-based overlay)
  - **In decode stage, 1.9x speedup for long output sequences and 5.7x more energy-efficient vs. NVIDIA A100 GPU**
  - Fewer than 50 lines of schedule code in Allo

A100 GPU *7nm*
Max M: 16415 (temporal)
Bandwidth: **1935 GB/s**

Alveo U280 (ours) *16nm*
Max M: 1289
Bandwidth: **498 GB/s**



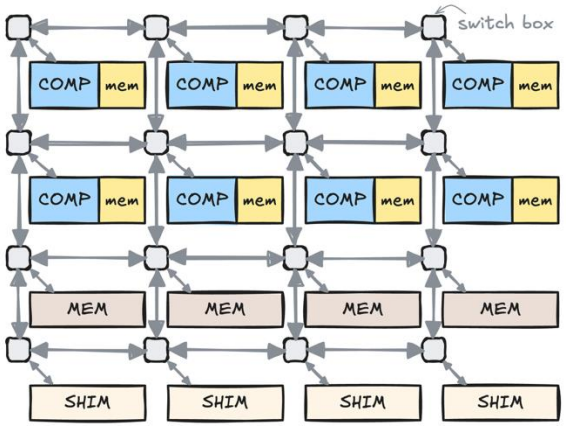
	Allo	DFX
Device	U280	U280
Freq.	250MHz	200MHz
Quant.	W4A8	fp16
BRAM	384 (19.0%)	1192 (59.1%)
DSP	1780 (19.73%)	3533 (39.2%)
FF	652K (25.0%)	1107K (42.5%)
LUT	508K (39.0%)	520K (39.9%)

H. Chen et al., "Understanding the Potential of FPGA-Based Spatial Acceleration for Large Language Model Inference", ACM TRETS, 2024.

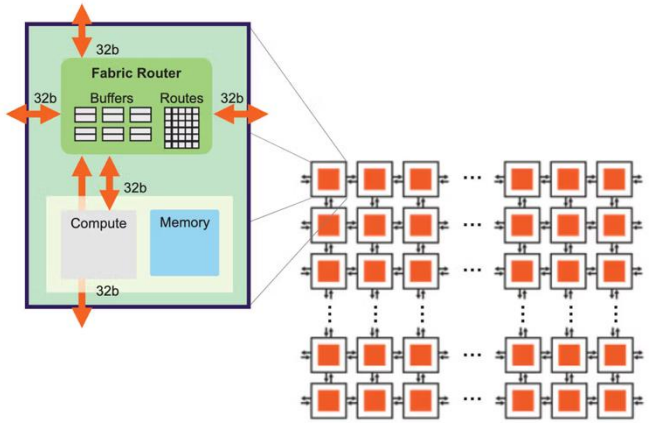
From Application-Specific Acceleration to Domain-Specific

- Dataflow accelerators are gaining attention
  - Examples: AMD AIE, Cerebras WSE, SambaNova RDU, Tenstorrent Wormhole, etc.

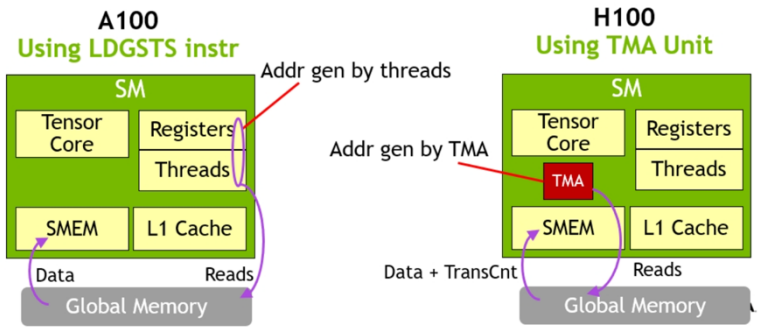
- GPUs are also embracing dataflow-style execution
  - via warp specialization (NVIDIA) / workgroup specialization (AMD)



AMD Ryzen NPU (with AIE)



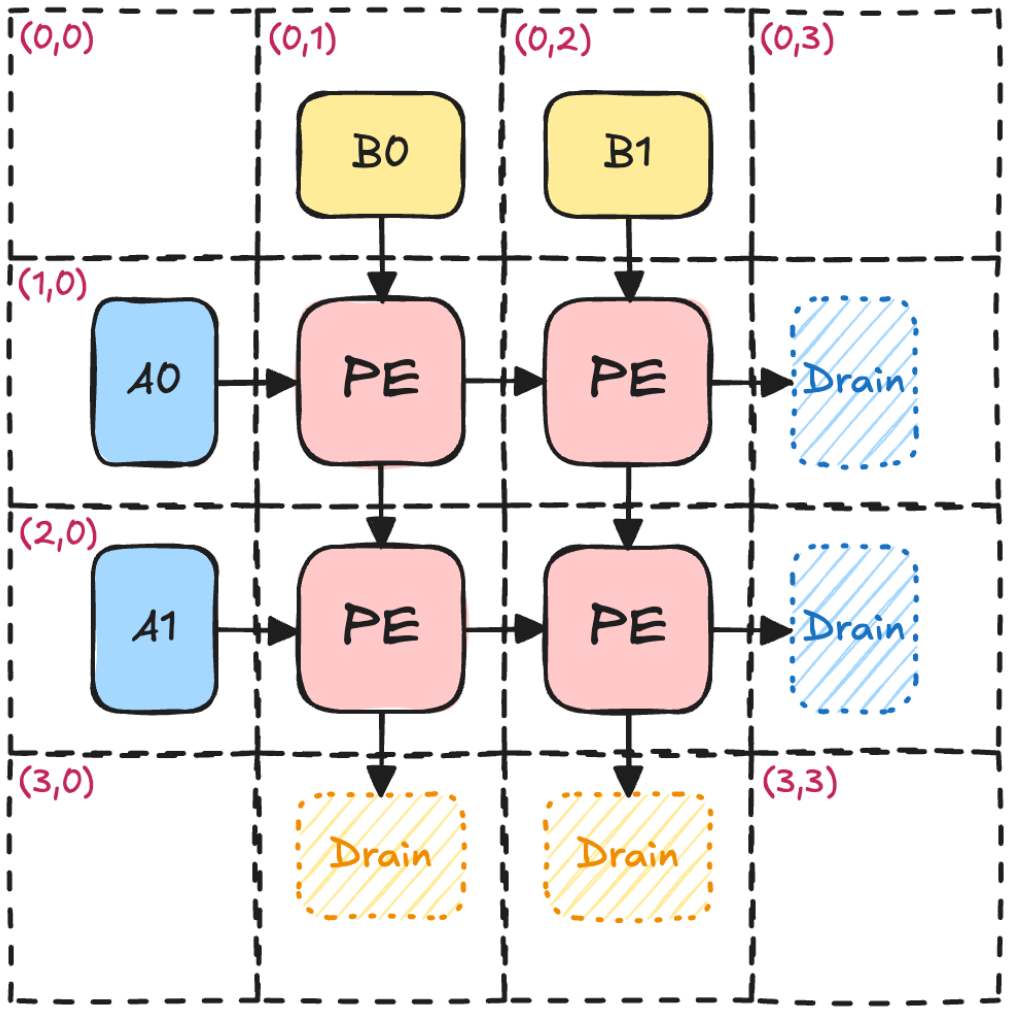
Cerebras Wafer-Scale Engine (WSE)



NVIDIA Tensor Memory Accelerator (TMA)

- Most accelerators use Single-Program Multiple-Data (SPMD) style to program
  - e.g., CUDA, Triton, JAX

From Application-Specific Acceleration to Domain-Specific



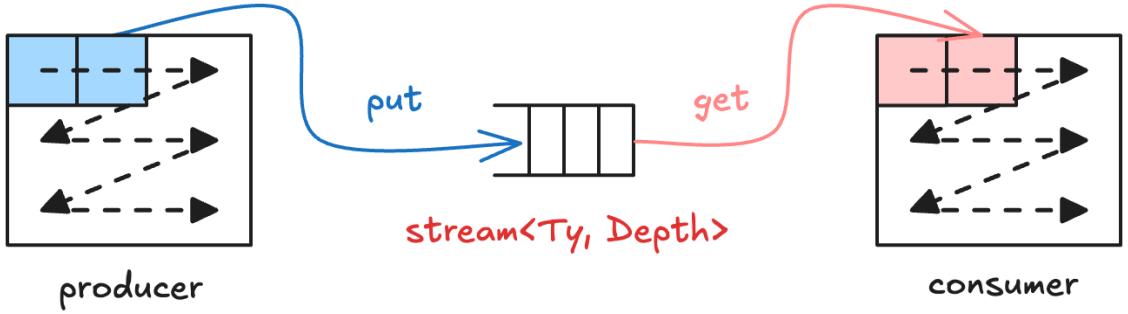
Natural question: Can SPMD used for modeling domain-specific accelerators?

Key insight: Accelerators are typically regular and spatially scalable by design, fitting naturally into a grid

Challenge: Compute units (PEs) are usually tightly interconnected, not independent workers

Allo Stream Type

- Data communication as a **first-class citizen**
  - Exposed in the frontend (different from aref)
  - Enabling compile-time type checking
  - e.g., dropped or double-consumed data, deadlocks, and mismatched access patterns



$$\frac{\Gamma \vdash s : \mathrm{Stream}[T, N] \qquad \Gamma \vdash x : T \qquad S = \mathrm{id}(s)}{\Gamma\ ;\ \Delta \uplus \{\, \mathrm{Free}(S) \,\} \ \vdash\ \mathrm{put}(s, x) : \mathbf{1}\ ;\ \Delta \uplus \{\, \mathrm{Used}(S) \,\}} \ (\text{T-Put})$$

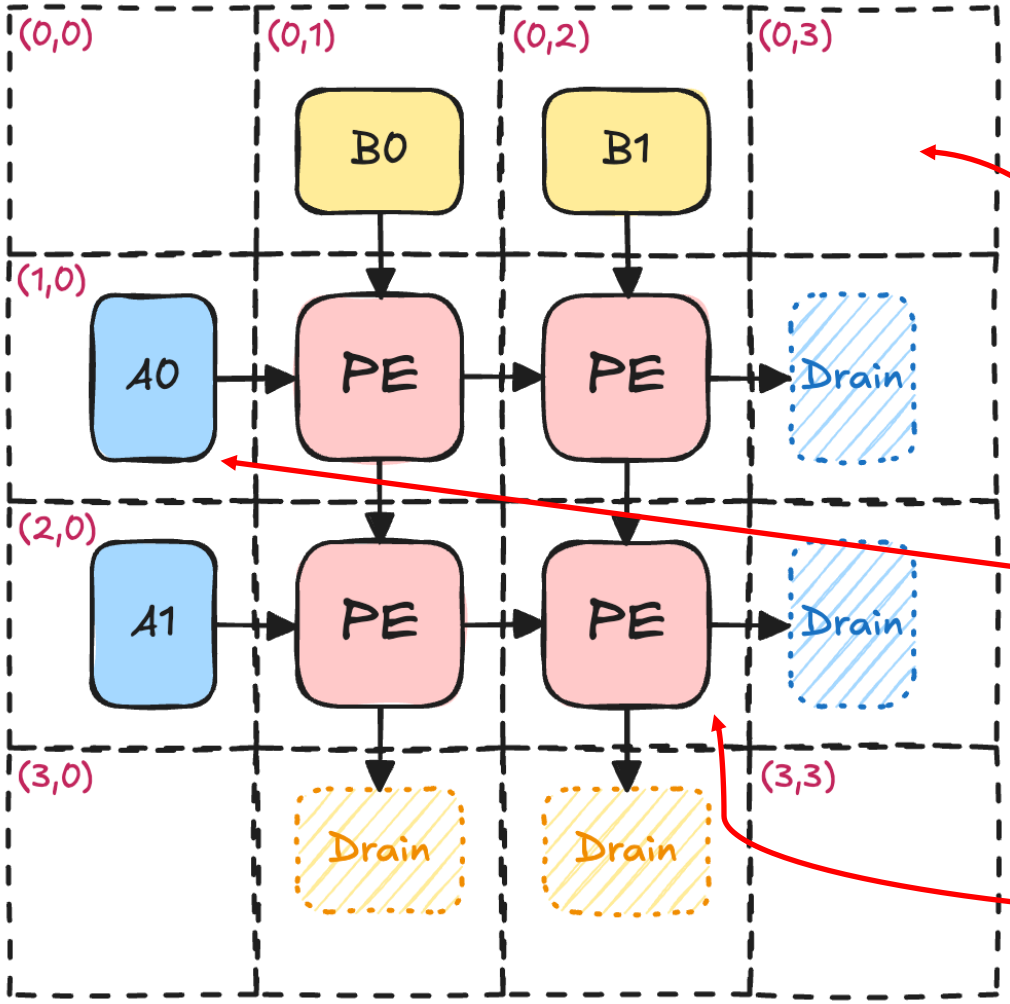
$$\frac{\Gamma \vdash s : \mathrm{Stream}[T, N] \qquad S = \mathrm{id}(s)}{\Gamma\ ;\ \Delta \uplus \{\, \mathrm{Used}(S) \,\} \ \vdash\ \mathrm{get}(s) : \mathrm{LFuture}\, T\ ;\ \Delta \uplus \{\, \mathrm{Free}(S) \,\}} \ (\text{T-Get})$$

$$\frac{}{\Gamma\ ;\ \Delta \uplus \{\, f : \mathrm{LFuture}\, T \,\} \ \vdash\ \mathrm{await}(f) : T\ ;\ \Delta} \ (\text{T-Await})$$

```
Ty = float32
M, N, K = 16, 16, 16

def top():
 pipe: Stream[Ty, 2]

 @df.work(grid=[1])
 def producer(A: Ty[M, N]):
  for i, j in allo.grid(M, N):
   # send data
   pipe.put(A[i, j])

 @df.work(grid=[1])
 def consumer(B: Ty[M, N]):
  for i, j in allo.grid(M, N):
   # receive data
   B[i, j] = pipe.get() + 1
```

S. Fang*, **H. Chen*** et al., "Dato: A Task-Based Programming Model for Dataflow Accelerators", arXiv, 2025.

SPMW: Single Program Multiple (Connected) Work Units




```
def top():
 fifo_A: Stream[int8, 4][P0, P1]
 fifo_B: Stream[int8, 4][P0, P1]

 @df.work(grid=[P0, P1])
 def gemm(A: int8[M, K],
          B: int8[K, N],
          C: int16[M, N]):
  i, j = df.get_wid()
  # peripheral kernels
  with allo.meta_if(i in {0, M + 1}
              and j in {0, N + 1}):
   pass
  with allo.meta_elif(j == 0):
   # i > 0
   for k in range(K):
    fifo_A[i, j + 1].put(A[i - 1, k])
  # ...
  # main body
  with allo.meta_else():
   c: int16 = 0
   for k in range(K):
    a: int8 = fifo_A[i, j].get()
    b: int8 = fifo_B[i, j].get()
    c += a * b
    fifo_A[i, j + 1].put(a)
    fifo_B[i + 1, j].put(b)
    C[i - 1, j - 1] = c
```

① Single program to describe **many work units over a grid**

② Type-safe specification of **connectivity among work units**

2D systolic array in Allo SPMW (36 lines)

SPMW Key Advantages

- SPMW is not just syntactic sugar*
  Direct encoding of regularity in the compiler IR (MLIR-based) for efficient analysis and optimization
- Explicit connectivity and modularity
  Simpler typing, verification, and physical design
- The "grid" composes hierarchically
  Easier IP integration; extensible to modeling RTL components

```
def top():
 fifo_A: Stream[int8, 4][P0, P1]
 fifo_B: Stream[int8, 4][P0, P1]

 @df.kernel(grid=[P0, P1])
 def gemm(A: int8[M, K],
          B: int8[K, N],
          C: int16[M, N]):
  i, j = df.get_pid()
  # peripheral kernels
  with allo.meta_if(i in {0, M + 1}
              and j in {0, N + 1}):
   pass
  with allo.meta_elif(j == 0):
   # i > 0
   for k in range(K):
    fifo_A[i, j + 1].put(A[i - 1, k])
  # ...
  # main body
  with allo.meta_else():
   c: int16 = 0
   for k in range(K):
    a: int8 = fifo_A[i, j].get()
    b: int8 = fifo_B[i, j].get()
    c += a * b
    fifo_A[i, j + 1].put(a)
    fifo_B[i + 1, j].put(b)
    C[i - 1, j - 1] = c
```

① Single program to describe **many work units over a grid**

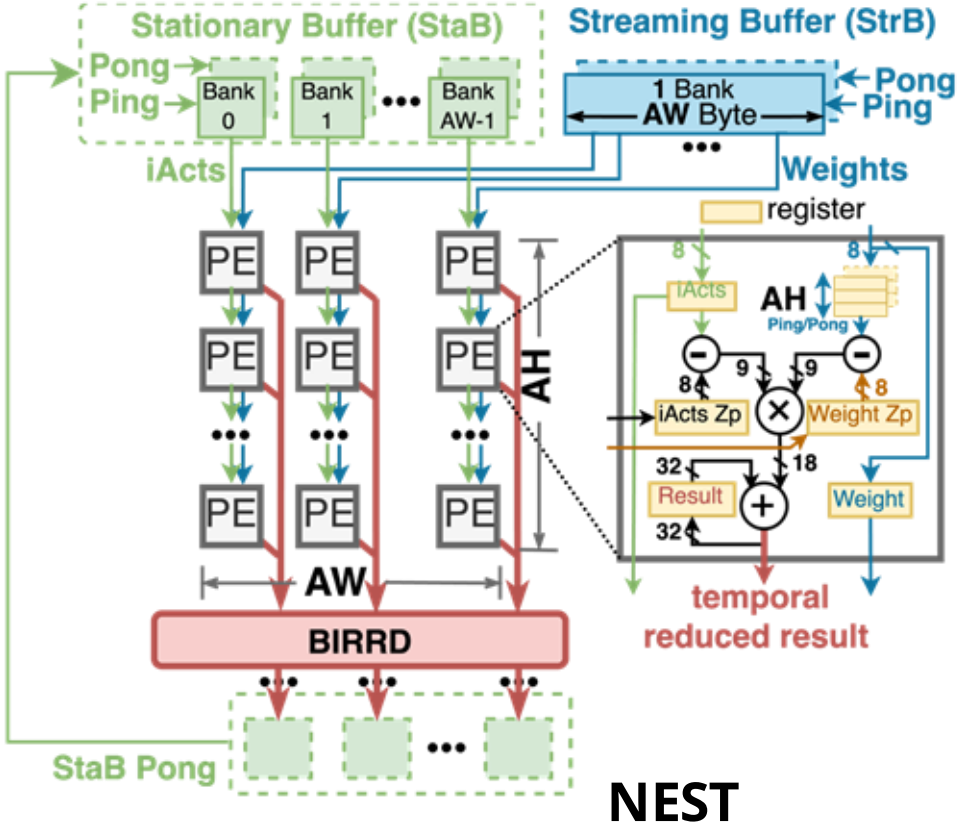
② Type-safe specification of **connectivity among work units**

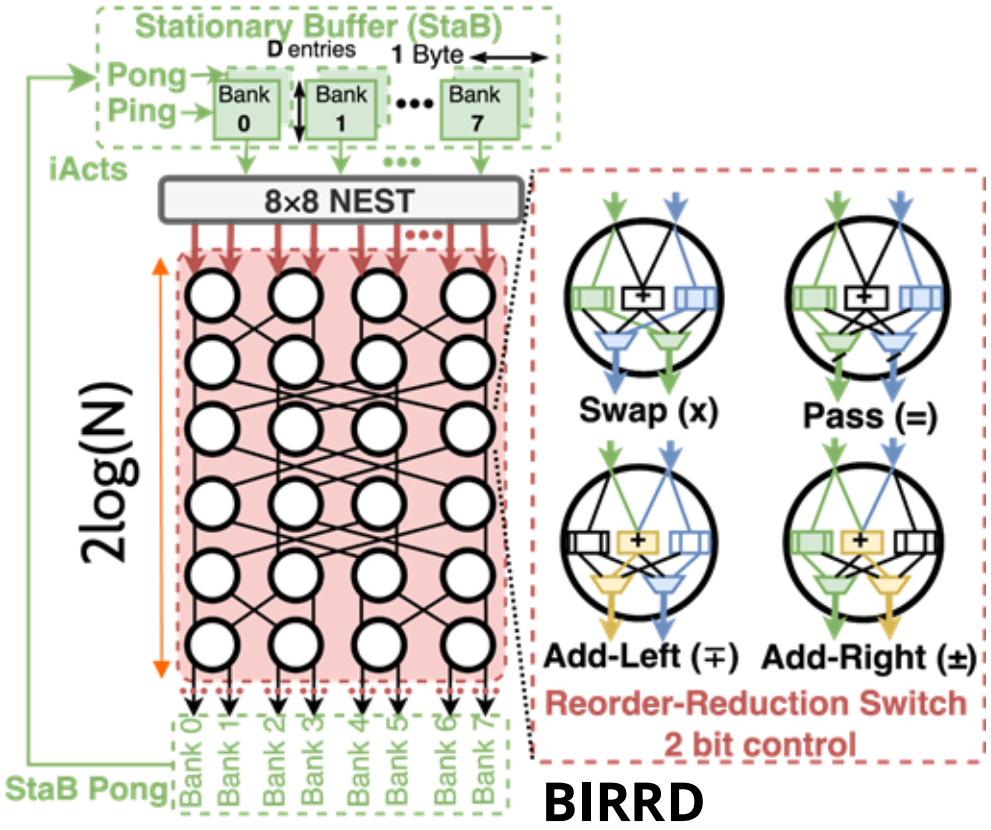
2D systolic array in Allo SPMW (36 lines)

* Metaprogramming tricks make code more compact but still bloat IR and offer little benefit for analysis

Case Study: FEATHER Tensor Accelerator

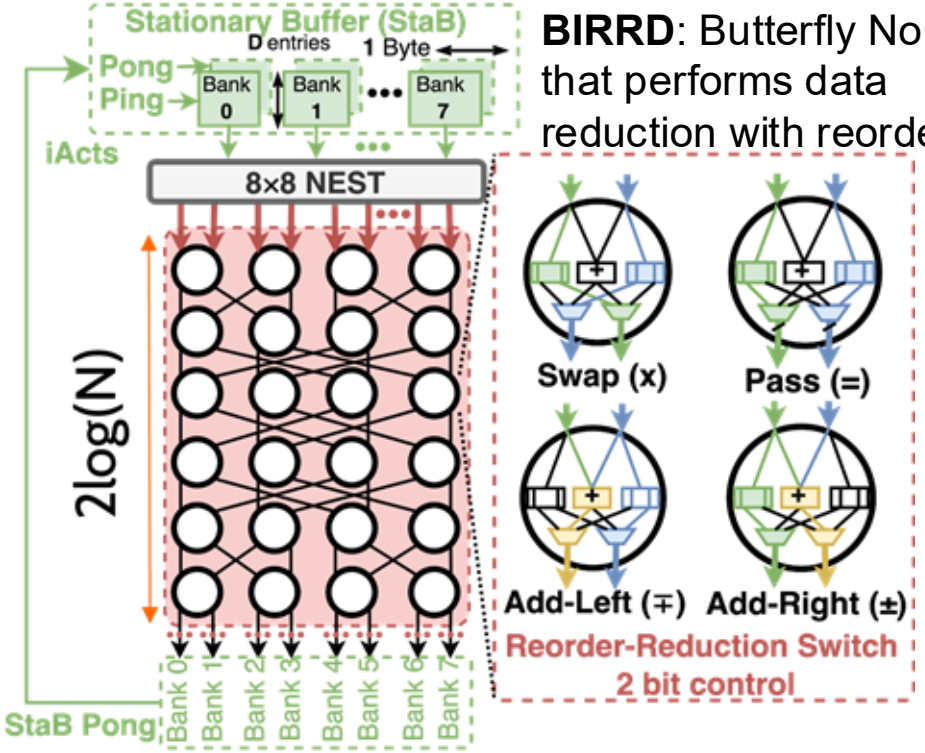
- **FEATHER** by Tushar's group at GT: A tensor DSA with reconfigurable co-switching of dataflows and data layouts
  - **NEST:** Compute engine with a spatial PE array
  - **BIRRD**: On-chip multi-stage butterfly network that performs data reduction with data reordering





J. Tong et al., "FEATHER: A Reconfigurable Accelerator with Data Reordering Support for Low-Cost On-Chip Dataflow Switching", ISCA 2024.

Case Study: FEATHER Tensor Accelerator



BIRRD: Butterfly NoC that performs data reduction with reordering

FEATHER in RTL: 2491 lines of Verilog (16x16 array)

```
@df.kernel(grid=[P0, P1])
def BIRRD():
 i, j = df.get_pid()
 inst = inst_input[i, j].get()

 mix = lambda L, R: [(L, R), (L, L+R), (L+R, R), (R, L)][inst]
 for _ in range(AH):
  if (i == 0):
   L, R = connection[0, 2*j].get(), connection[0, 2*j+1].get()
   a, b = mix(L, R)
   connection[i+1, reverse_bits(2*j, 2)].put(a)
   connection[i+1, reverse_bits(2*j+1, 2)].put(b)
  elif (i == P0-1):
   L, R = connection[P0-1, 2*j].get(), connection[P0-1, 2*j+1].get()
   a, b = mix(L, R)
   connection[P0, 2*j].put(a)
   connection[P0, 2*j+1].put(b)
  else:
   L, R = connection[i, 2*j].get(), connection[i, 2*j+1].get()
   a, b = mix(L, R)
   w = min(LOG2_AW, 2+i, 2*LOG2_AW-i)
   connection[i+1, reverse_bits(2*j, w)].put(a)
   connection[i+1, reverse_bits(2*j+1, w)].put(b)
```

Single Program, Multiple connected Work Units (**SPMW**), over a 2D grid

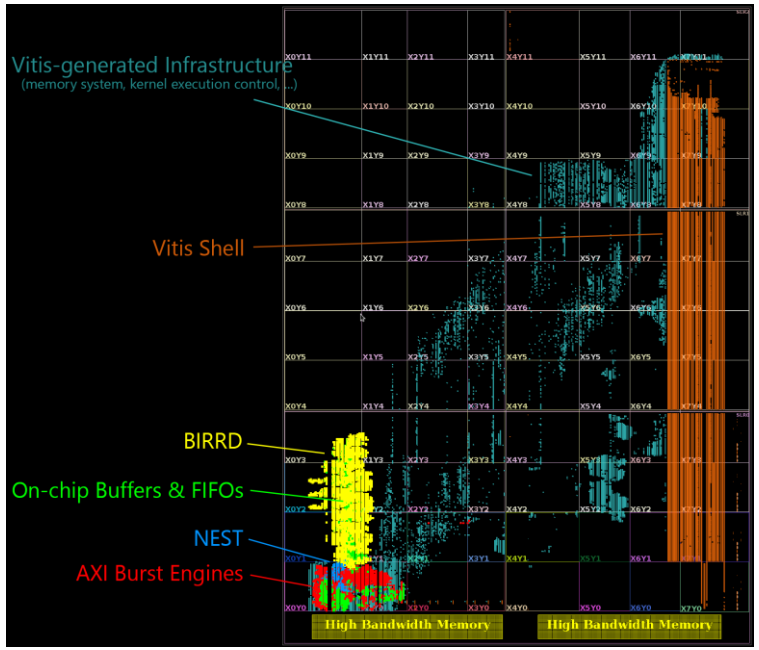
Type-safe specification of interconnections among work units

FEATHER in Allo: 220 lines of Python (same performance, lower area)

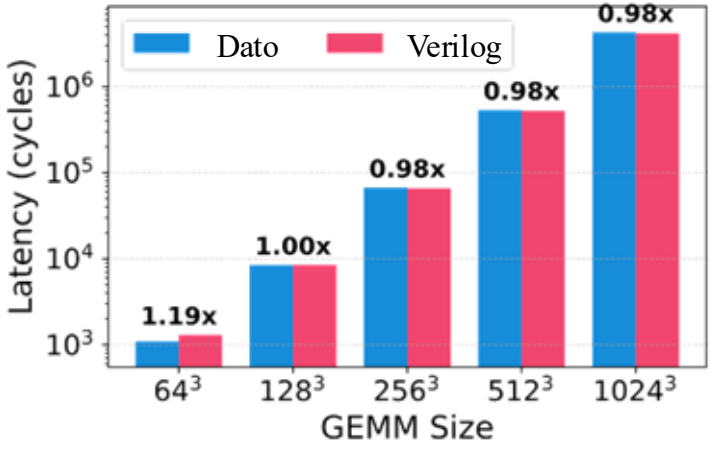
[1] J. Tong, et al. "FEATHER: A reconfigurable accelerator with data reordering support for low-cost on-chip dataflow switching," ISCA 2024.

Case Study: FEATHER Tensor Accelerator

- Allo provides **SPMW, a grid-based abstraction** that explicitly captures regularity
- Line-of-code comparison
  - Verilog RTL: 2941 lines
  - Allo: **220** lines
- Comparable performance
  - 16x16 PE array
  - FEATHER RTL: 311 cycles
  - Allo-FEATHER: **301** cycles



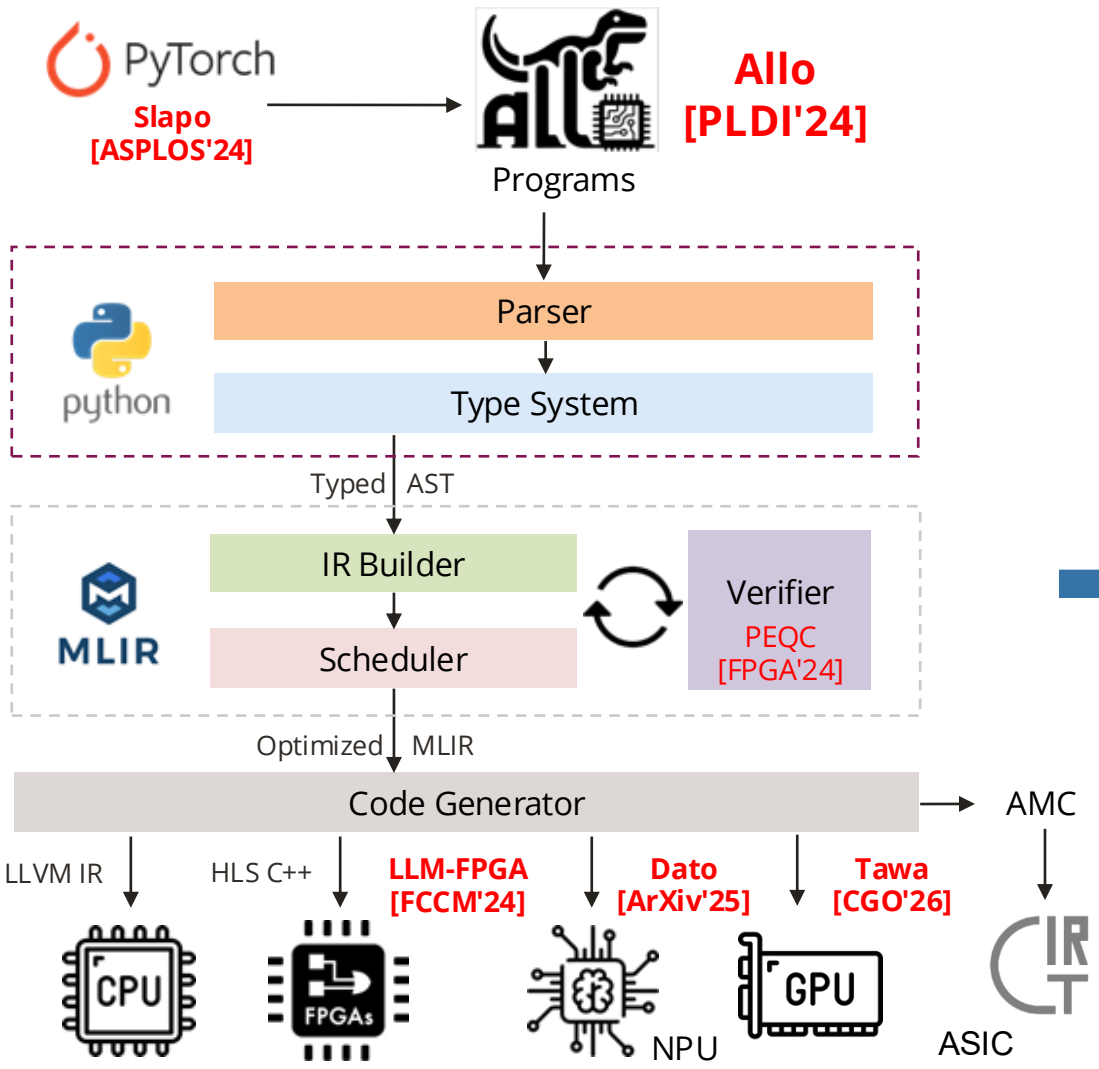
(a) Allo-Generated FEATHER on U280



(b) Customizable Template

	LUT	FF	BRAM	DSP
Allo-FEATHER	**29181**	**35808**	**2**	0
FEATHER RTL	50434	82815	30	0

Allo Summary



cornell-zhang/**allo**

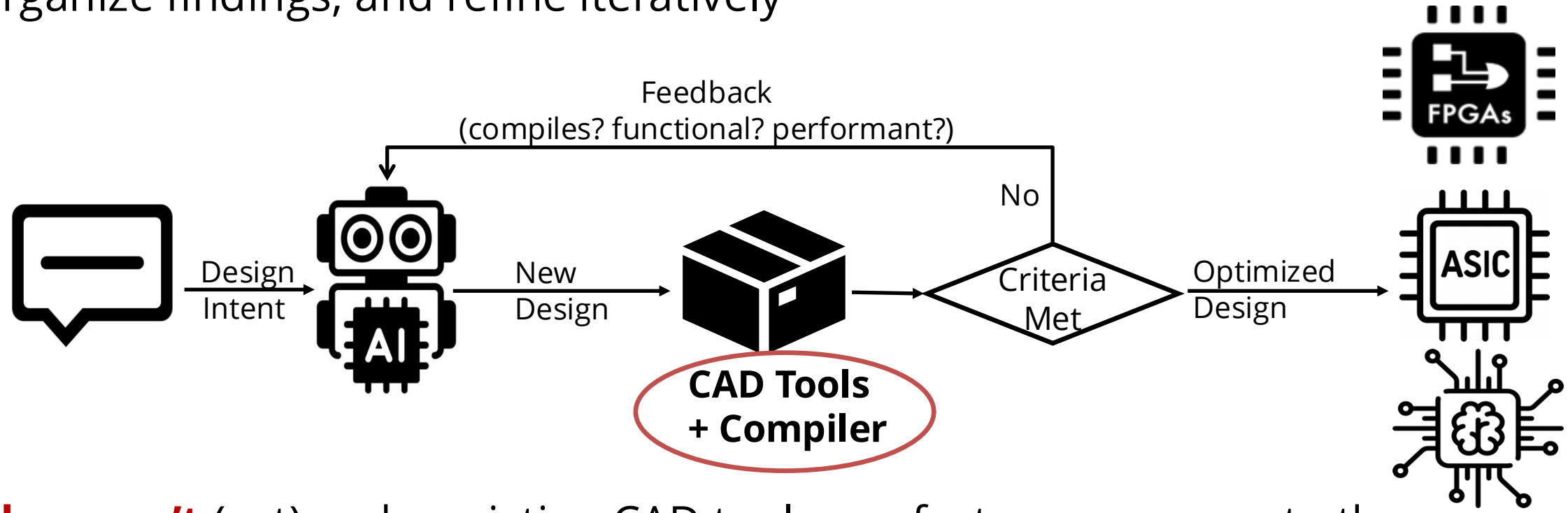
Allo Accelerator Design and Programming Framework (PLDI'24)

https://github.com/cornell-zhang/allo

38 Contributors | 55 Issues | 29 Discussions | 360 Stars | 67 Forks

Agentic AI for Accelerator Design?

Agents can write code (HDL included), set up environments, run tests, organize findings, and refine iteratively



They can't (yet) make existing CAD tools run faster, nor generate the compilation stack needed to program the new accelerators

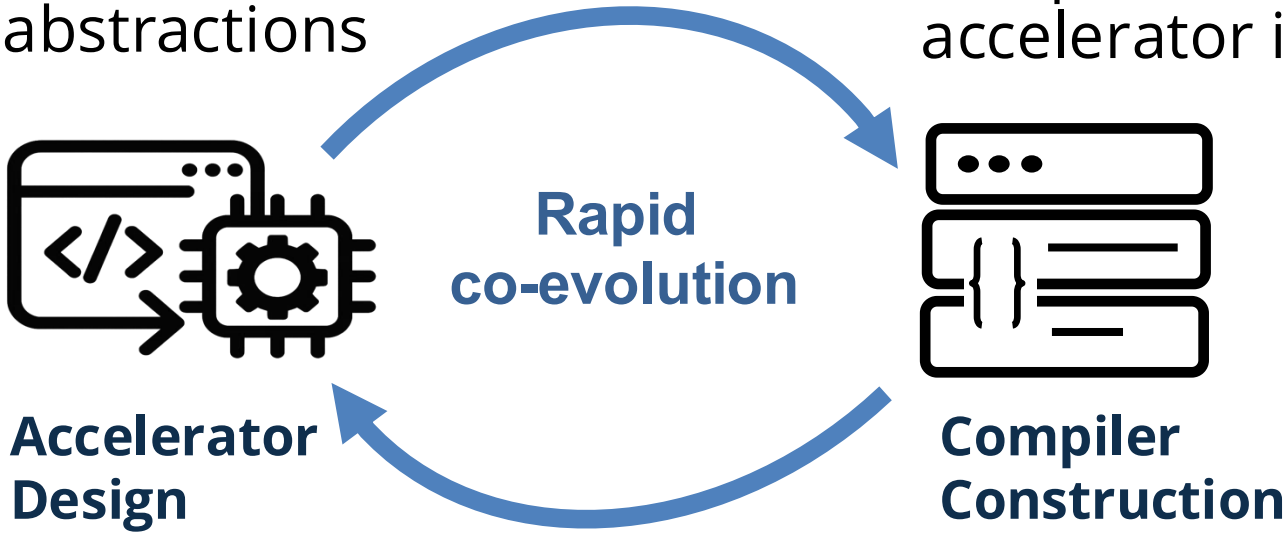
agent performance ≈ reasoning × number of iterations

Agents are at their best when they can iterate quickly through propose–test–refine loops

Speeding Up the Accelerator Design Loop

Rapid accelerator modeling & generation by raising abstractions

Quick bring-up of compiler stack for new accelerator instances



<u>Our approaches</u>

1. **Unifying abstractions for accelerator design & programming**
2. **Leveraging modern AI systems across both**

Disconnected Abstractions in Accelerator Design & Programming

Hardware Description Languages

Common entries: `(System)Verilog, VHDL, Bluespec, Chisel`

Primary abstraction: **RTL, behavior/structural**

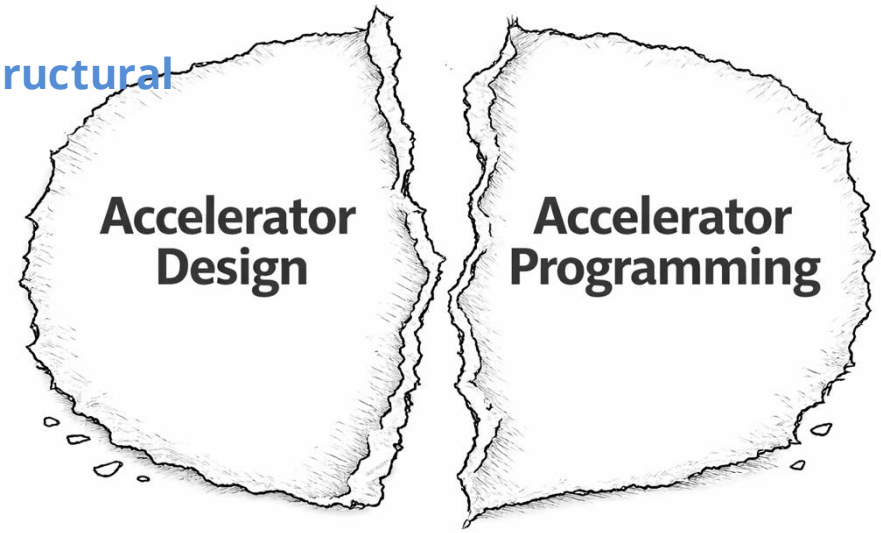
Mental model: explicit datapath+FSM

High-Level Synthesis (HLS)

Common entries: `C/C++, SystemC, XLS, and various HLS DSLs`

Primary abstraction: **sequential loop-based code with pragmas**

Mental model: implicit pipelined/parallelized architecture inferred from sequential loops



(Performant) Programming for AI Hardware

Common entries: `CUDA/CUTLASS, HIP/ROCM, SYCL/oneAPI, OpenCL, Triton`

Primary abstraction: single program, multiple data (SPMD)

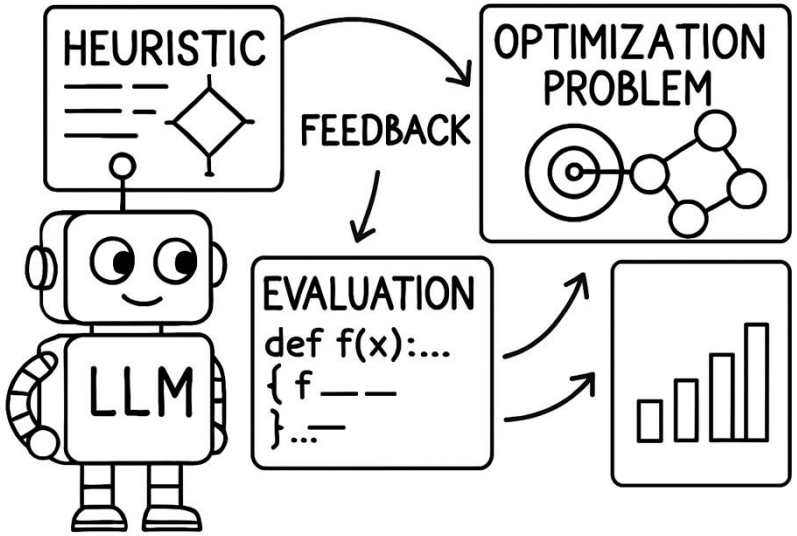
Mental model: many (large) independent threads over a grid

Vibe Coding?

Abstractions matter even more to speed up agentic iterations and reduce token costs

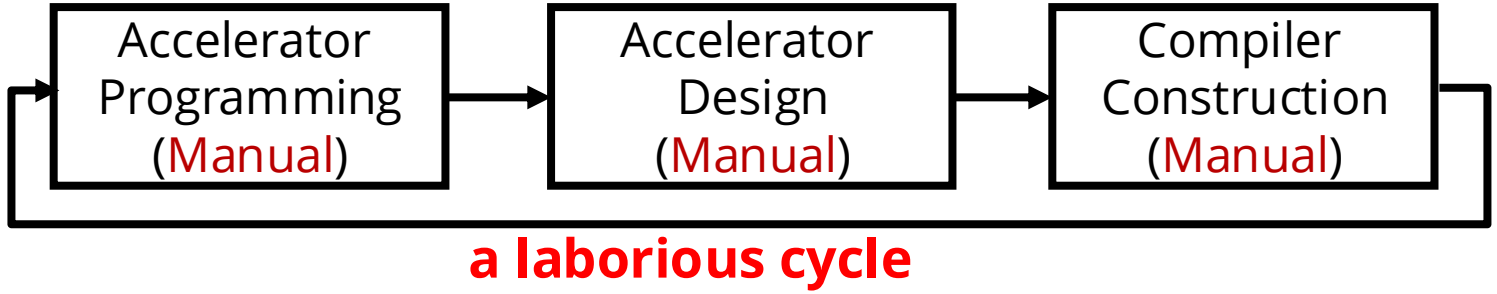
Making Accelerator Compiler Development Easier

- Allo makes it easier to build and program accelerators
- But the Allo compiler itself is still manually constructed
- **Can we leverage modern AI systems to further simplify accelerator compiler development?**

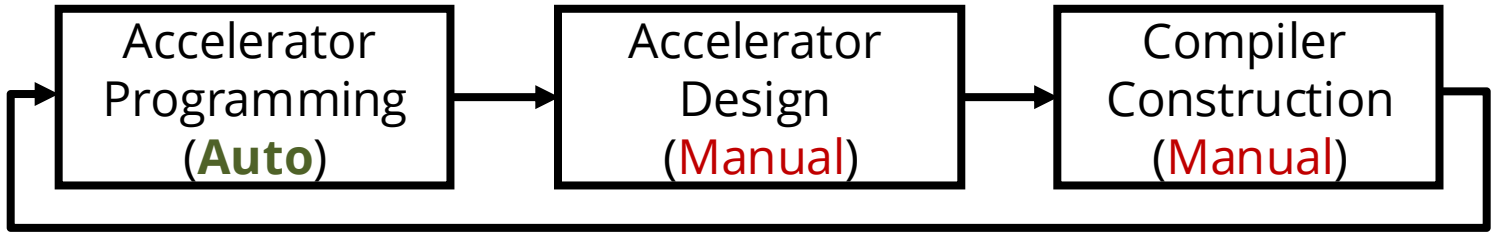
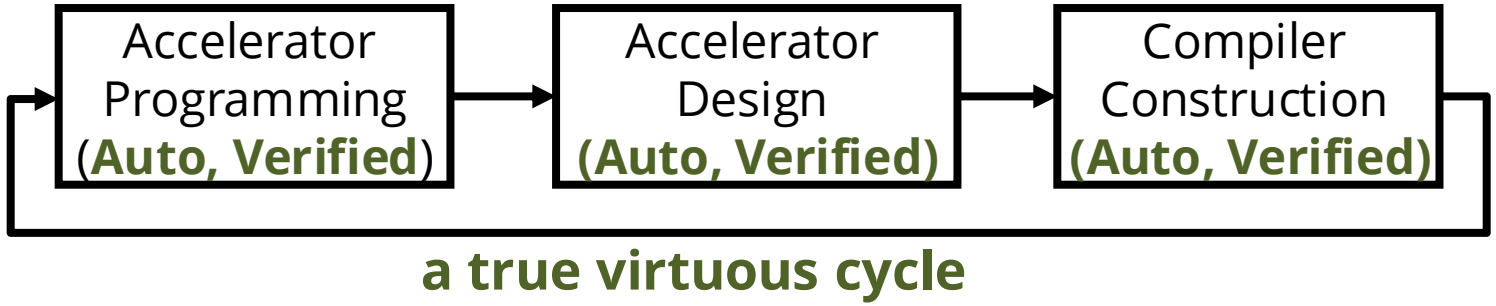


Hypothesizing (Fantasizing) Autonomous Accelerator Design

Where are we now



Where are we heading in the near future



What could be the next big game changer